Задачи к экзамену по дисциплине

Конструирование алгоритмов и структур данных

Задачи на алгоритмы

1. Дан массив целых чисел длины n, который может содержать отрицательные значения и повторяющиеся элементы. Требуется за один проход по массиву найти минимальный элемент, его количество и индекс самого левого вхождения, а затем обосновать, почему для любой реализации этой задачи в модели сравнения нижняя оценка по времени составляет $\Omega(n)$.

2. Даны два неубывающих массива a и b одинаковой длины n. Требуется за $O(n)$ найти такое значение S, что существует пара индексов i и j с $a[i]+b[j]=S$, причём S заранее неизвестно и выбирается так, чтобы количество подходящих пар было максимальным, после чего нужно вывести это S и число найденных пар.

3. Дан массив длины n. Пара индексов i и j называется инверсией, если i меньше j и $a[i]$ больше $a[j]$. Требуется посчитать количество инверсий за $O(n \cdot \log n)$, причём в ответе нужно вывести и число инверсий, и отсортированный массив, используя модификацию сортировки слиянием без дополнительных проходов по данным.

4. Дана последовательность n операций вида добавить число x или извлечь минимум. Требуется реализовать обработку операций на структуре бинарная куча так, чтобы каждая операция работала за $O(\log n)$, а также дополнительно поддержать операцию уменьшить ключ элемента по его текущему индексу и корректно восстановить свойство кучи после изменения.

5. Дан массив a длины n и целое число k. Требуется найти k-ю порядковую статистику, то есть элемент, который стоял бы на позиции k в отсортированном массиве, используя рандомизированный быстрый выбор, а затем подобрать тесты, на которых фиксированный выбор опорного элемента приводит к квадратичному времени, и объяснить, почему рандомизация защищает от таких тестов в среднем.

6. Дан отсортированный массив и большое количество запросов вида найти нижнюю границу значения x и верхнюю границу значения x. Требуется реализовать обе операции бинарным поиском так, чтобы они корректно работали при наличии повторов, и на каждом запросе возвращали минимальный индекс элемента не меньше x и минимальный индекс элемента больше x.

7. Дана функция проверки, можно ли произвести x изделий при заданных ресурсах и правилах обмена ресурсов на универсальную валюту. Требуется найти максимальное x, которое можно произвести, используя бинарный поиск по ответу, и аккуратно сформулировать монотонное свойство, на котором основан поиск, включая обработку больших значений и переполнений.

8. Требуется реализовать очередь на двух стеках так, чтобы операции добавить в конец и извлечь из начала работали корректно для любой последовательности вызовов. Дополнительно нужно провести амортизированный анализ и показать, что средняя стоимость операции на длинной последовательности вызовов составляет $O(1)$, несмотря на возможные дорогие переносы элементов.

9. Дан набор элементов, изначально каждый в своём множестве, и последовательность операций объединить множества и проверить, принадлежат ли два элемента одному множеству. Требуется реализовать систему непересекающихся множеств с ранговой эвристикой и сжатием путей, а затем на серии операций объяснить, почему амортизированная сложность близка к константной и откуда появляется обратная функция Аккермана.

10. Даны веса предметов и их стоимости, а также ограничение на вместимость рюкзака. Требуется решить задачу о рюкзаке 0/1 методом динамического программирования, оптимизировав память до одного массива, и дополнительно восстановить один из оптимальных наборов предметов, не используя полную таблицу.

11. Дан массив чисел длины n и множество запросов двух типов: изменить значение элемента по индексу и получить ответ на отрезке. Требуется реализовать дерево отрезков, которое на запросе возвращает минимум на отрезке и количество элементов, равных этому минимуму, а на обновлении изменяет один элемент, сохраняя корректность агрегатов.

12. Дан массив длины n и требуется обрабатывать запросы на сумму на префиксе и точечное обновление значения. Требуется реализовать дерево Фенвика, пояснив, как устроены переходы по индексу через операцию $i \& -i$, и дополнительно реализовать поиск наибольшего префикса, на котором сумма не превышает заданного x при условии неотрицательных элементов.

13. Дано дерево из n вершин с корнем и множество запросов вида найти наименьшего общего предка двух вершин. Требуется реализовать метод бинарных подъёмов, описать необходимую предобработку, обеспечить ответ за $O(\log n)$ и корректно обработать случай, когда одна вершина является предком другой.

14. Дан ориентированный граф. Требуется найти компоненты сильной связности, построить конденсацию графа и затем по конденсации определить минимальное число дуг, которые нужно добавить, чтобы исходный граф стал сильно связным, корректно обрабатывая случаи, когда компонент одна.

15. Дан связный неориентированный граф. Требуется найти все мосты и точки сочленения за линейное время, затем по найденным мостам определить, сколько рёбер нужно добавить, чтобы граф стал рёберно двусвязным, и предъявить любой корректный способ добавления.

16. Дана пара строк s и t произвольной длины, состоящих из латинских букв в нижнем регистре. Требуется вычислить расстояние Левенштейна между s и t методом динамического программирования, построив таблицу минимального числа операций вставки, удаления и замены, необходимых для преобразования одной строки в другую, а затем вывести само расстояние и одну корректную последовательность операций, которая его достигает.

17. Дана пара строк s и t произвольной длины. Требуется реализовать вычисление расстояния Левенштейна по динамике так, чтобы память была оптимизирована до двух строк таблицы, при этом результат должен совпадать с полным двумерным вычислением, а также нужно пояснить, какие значения теряются при таком сжатии и почему восстановление последовательности операций становится нетривиальным.

18. Дана строка s и целое число d. Требуется по динамическому программированию определить, существует ли строка t, отличающаяся от s по расстоянию Левенштейна не более чем на d, которая удовлетворяет заданному набору ограничений на позиции символов, и описать, как ограничение d позволяет отсекать вычисления и ускорять работу по сравнению с полным заполнением таблицы.

19. Дана тропинка из n клеток, пронумерованных от 1 до n, и кузнечик стартует в клетке 1 и должен попасть в клетку n. За один прыжок кузнечик может переместиться вперёд на 1, 2, …, k клеток, при этом у каждой клетки i задана стоимость $a[i]$, которую кузнечик платит при приземлении в эту клетку. Требуется найти минимальную суммарную стоимость пути до клетки n и восстановить один из оптимальных маршрутов, используя динамическое программирование за $O(n \cdot k)$.

20. Дана тропинка из n клеток, пронумерованных от 1 до n, и кузнечик стартует в клетке 1 и должен попасть в клетку n. За один прыжок кузнечик может переместиться вперёд на 1, 2, …, k клеток, при этом у каждой клетки i задана стоимость $a[i]$, которую кузнечик платит при приземлении. Требуется найти минимальную стоимость достижения каждой клетки и минимальную стоимость пути до клетки n быстрее, чем за $O(n \cdot k)$, используя динамическое программирование с оптимизацией переходов через структуру данных, которая поддерживает удаление элементов, выходящих из окна длины k, и получение текущего минимума, и добиться асимптотики $O(n \cdot \log k)$.

Задачи на структуры данных

1. Создайте обобщённый класс MyArrayList для динамического массива, который хранит элементы во внутреннем массиве и увеличивает ёмкость при переполнении. Реализуйте добавление в конец, вставку по индексу через индексатор, удаление по индексу, поиск индекса первого вхождения, а также методы Equals, GetHashCode и ToString так, чтобы два списка считались равными при равенстве размеров и попарном равенстве элементов.

2. Создайте обобщённый класс MyLinkedList двусвязного списка с узлами, содержащими значение и ссылки на предыдущий и следующий узел. Реализуйте добавление в конец, добавление в начало, удаление первого вхождения заданного значения, доступ к элементу по индексу через индексатор с проходом от ближайшего конца, а также методы Equals, GetHashCode и ToString с понятным строковым представлением последовательности.

3. Создайте обобщённый класс MyStack на основе массива или вектора с операциями push, pop, peek и свойством Count. Дополнительно перегрузите операторы равенства и неравенства для сравнения стеков по содержимому от дна к вершине, реализуйте Equals и GetHashCode согласованно с перегрузкой операторов, а ToString сделайте так, чтобы вершина стека была визуально выделена в выводе.

4. Создайте обобщённый класс MyQueue с кольцевым буфером и операциями Enqueue, Dequeue, Peek и свойством Count. Добавьте индексатор для чтения элемента по логическому индексу от головы очереди, реализуйте Equals и GetHashCode так, чтобы равные очереди имели одинаковые хеши, и обеспечьте ToString, показывающий порядок элементов от головы к хвосту.

5. Создайте обобщённый класс MyArrayDeque двунаправленной очереди на массиве, поддерживающий добавление и удаление с обоих концов. Реализуйте AddFirst, AddLast, RemoveFirst, RemoveLast и индексатор доступа по позиции относительно головы, а также Equals, GetHashCode и ToString, чтобы можно было удобно проверять корректность после серии смешанных операций.

6. Создайте обобщённый класс BinaryHeap с возможностью работать как min-куча или max-куча на основе компаратора. Реализуйте Insert, Peek, Extract, ChangeKey по индексу и индексатор для чтения элемента по внутреннему индексу, затем перегрузите оператор плюс так, чтобы он создавал новую кучу, содержащую элементы обеих куч, и добавьте Equals, GetHashCode и ToString для сравнения и отладки.

7. Создайте класс HashTable для хранения пар ключ значение, где ключ – это строка, а значение это целое число, с разрешением коллизий методом цепочек. Реализуйте Add или Update, TryGetValue, Remove, индексатор доступа по ключу и перечисление всех ключей, а также Equals, GetHashCode и ToString, где равенство означает одинаковое множество пар независимо от порядка в цепочках.

8. Создайте структуру Rational для рациональных чисел с числителем и знаменателем, всегда хранящихся в сокращённом виде и со знаком только в числителе. Реализуйте перегрузку операторов сложения, вычитания и умножения, сравнение через IComparable, Equals и GetHashCode, а также ToString в виде корректной дроби, плюс индексатор, который по индексу 0 возвращает числитель, а по индексу 1 возвращает знаменатель.

9. Создайте класс Matrix для целочисленной матрицы фиксированного размера, хранящей элементы в двумерном массиве. Реализуйте индексатор по двум индексам для чтения и записи, перегрузку оператора умножения матриц, метод вычисления транспонированной матрицы, а также Equals, GetHashCode и ToString так, чтобы матрицы сравнивались по размерам и всем элементам и красиво печатались строками.

10. Создайте класс Polynomial для многочлена с целыми коэффициентами, где коэффициенты хранятся по степеням. Реализуйте индексатор по степени, перегрузку операторов сложения и умножения многочленов, метод вычисления значения в точке, а также Equals, GetHashCode и ToString, выводящий многочлен в стандартном математическом виде без лишних нулевых слагаемых.

Задачи на ООП

1. Создайте иерархию классов для моделирования различных видов животных (не менее 3 дочерних классов) с базовым классом Animal. Базовый класс должен содержать общие свойства, такие как имя, возраст и вес, а также виртуальные методы для издавания звуков и движения. Дочерние классы (например, кошка, собака и птица) должны иметь свои специфические свойства и реализовывать методы базового класса. Попробуйте заменить базовый класс интерфейсом. Если задача позволяет, разбейте его на два интерфейса, чтобы разделить функциональность. Напишите программу, демонстрирующую работу всех классов и интерфейсов, включая создание объектов дочерних классов и вызов их методов.

2. Разработайте иерархию классов для представления различных видов музыкальных инструментов (не менее 3 дочерних классов) с базовым классом Instrument. Базовый класс должен содержать общие свойства, такие как название и тип инструмента, а также виртуальные методы для воспроизведения звука и настройки инструмента. Дочерние классы (например, гитара, фортепиано и скрипка) должны иметь свои специфические свойства и реализовывать методы базового класса. Попробуйте заменить базовый класс интерфейсом. Если задача позволяет, разбейте его на два интерфейса, чтобы разделить функциональность. Напишите программу, демонстрирующую работу всех классов и интерфейсов, включая создание объектов дочерних классов и вызов их методов.

3. Создайте иерархию классов для моделирования различных видов растений (не менее 3 дочерних классов) с базовым классом Plant. Базовый класс должен содержать общие свойства, такие как название, высота и тип растения, а также виртуальные методы для полива и удобрения. Дочерние классы (например, дерево, цветок и кустарник) должны иметь свои специфические свойства и реализовывать методы базового класса. Попробуйте заменить базовый класс интерфейсом. Если задача позволяет, разбейте его на два интерфейса, чтобы разделить функциональность. Напишите программу, демонстрирующую работу всех классов и интерфейсов, включая создание объектов дочерних классов и вызов их методов.

4. Разработайте иерархию классов для представления различных видов одежды (не менее 3 дочерних классов) с базовым классом Clothing. Базовый класс должен содержать общие свойства, такие как размер, цвет и материал, а также виртуальные методы для стирки и глажки. Дочерние классы (например, рубашка, брюки и платье) должны иметь свои специфические свойства и реализовывать методы базового класса. Попробуйте заменить базовый класс интерфейсом. Если задача позволяет, разбейте его на два интерфейса, чтобы разделить функциональность. Напишите программу, демонстрирующую работу всех классов и интерфейсов, включая создание объектов дочерних классов и вызов их методов.

5. Создайте иерархию классов для моделирования различных видов электронных устройств (не менее 3 дочерних классов) с базовым классом Device. Базовый класс должен содержать общие свойства, такие как марка, модель и потребляемая мощность, а также виртуальные методы для включения и выключения устройства. Дочерние классы (например, смартфон, ноутбук и телевизор) должны иметь свои специфические свойства и реализовывать методы базового класса. Попробуйте заменить базовый класс интерфейсом. Если задача позволяет, разбейте его на два интерфейса, чтобы разделить функциональность. Напишите программу, демонстрирующую работу всех классов и интерфейсов, включая создание объектов дочерних классов и вызов их методов.

6. Разработайте иерархию классов для представления различных видов геометрических тел (не менее 3 дочерних классов) с базовым классом Solid. Базовый класс должен содержать общие свойства, такие как объём и площадь поверхности, а также виртуальные методы для вычисления этих свойств. Дочерние классы (например, куб, сфера и цилиндр) должны иметь свои специфические свойства и реализовывать методы базового класса. Попробуйте заменить базовый класс интерфейсом. Если задача позволяет, разбейте его на два интерфейса, чтобы разделить функциональность. Напишите программу, демонстрирующую работу всех классов и интерфейсов, включая создание объектов дочерних классов и вызов их методов.

7. Создайте иерархию классов для моделирования различных видов банковских счетов (не менее 3 дочерних классов) с базовым классом Account. Базовый класс должен содержать общие свойства, такие как номер счёта и баланс, а также виртуальные методы для пополнения и снятия средств. Дочерние классы (например, сберегательный счёт, текущий счёт и кредитный счёт) должны иметь свои специфические свойства и реализовывать методы базового класса. Попробуйте заменить базовый класс интерфейсом. Если задача позволяет, разбейте его на два интерфейса, чтобы разделить функциональность. Напишите программу, демонстрирующую работу всех классов и интерфейсов, включая создание объектов дочерних классов и вызов их методов.

8. Разработайте иерархию классов для представления различных видов книг (не менее 3 дочерних классов) с базовым классом Book. Базовый класс должен содержать общие свойства, такие как название, автор и количество страниц, а также виртуальные методы для чтения и поиска информации. Дочерние классы (например, художественная книга, учебник и энциклопедия) должны иметь свои специфические свойства и реализовывать методы базового класса. Попробуйте заменить базовый класс интерфейсом. Если задача позволяет, разбейте его на два интерфейса, чтобы разделить функциональность. Напишите программу, демонстрирующую работу всех классов и интерфейсов, включая создание объектов дочерних классов и вызов их методов.

9. Создайте иерархию классов для моделирования различных видов спортивных игр (не менее 3 дочерних классов) с базовым классом Game. Базовый класс должен содержать общие свойства, такие как название игры и количество игроков, а также виртуальные методы для начала и окончания игры. Дочерние классы (например, футбол, баскетбол и теннис) должны иметь свои специфические свойства и реализовывать методы базового класса. Попробуйте заменить базовый класс интерфейсом. Если задача позволяет, разбейте его на два интерфейса, чтобы разделить функциональность. Напишите программу, демонстрирующую работу всех классов и интерфейсов, включая создание объектов дочерних классов и вызов их методов.

10. Разработайте иерархию классов для представления различных видов музыкальных жанров (не менее 3 дочерних классов) с базовым классом MusicGenre. Базовый класс должен содержать общие свойства, такие как название жанра и характерные инструменты, а также виртуальные методы для воспроизведения музыки и описания жанра. Дочерние классы (например, рок, джаз и классическая музыка) должны иметь свои специфические свойства и реализовывать методы базового класса. Попробуйте заменить базовый класс интерфейсом. Если задача позволяет, разбейте его на два интерфейса, чтобы разделить функциональность. Напишите программу, демонстрирующую работу всех классов и интерфейсов, включая создание объектов дочерних классов и вызов их методов.